The Journal of Open Source Software

Adaptive Resonance Lib: A Python package for Adaptive Resonance Theory (ART) models

Niklas M. Melton [1¶], **Dustin Tanksley** [1], and **Donald C. Wunsch II** [1]

1 Missouri University of Science and Technology, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 15 November 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The Adaptive Resonance Theory Library (**artlib**) is a Python library that implements a wide range of Adaptive Resonance Theory (ART) algorithms. **artlib** currently supports eight elementary ART models and 11 compound ART models. These models can be applied to tasks such as unsupervised clustering, supervised classification, regression, and reinforcement learning (Silva et al., 2019a). This library provides an extensible and modular framework where users can integrate custom models or extend current implementations, allowing for experimentation with existing and novel machine learning techniques.

In addition to the diverse set of ART models, **artlib** offers implementations of visualization methods for various cluster geometries, along with pre-processing techniques such as Visual Assessment of Tendency (VAT) (Bezdek & Hathaway, 2002), data normalization, and complement coding.

Adaptive Resonance Theory (ART)

ART is a class of neural networks known for solving the stability-plasticity dilemma, making it particularly effective for classification, clustering, and incremental learning tasks (Grossberg, 1976, 1976, 1980, 2013; Silva et al., 2019a). ART models are designed to dynamically learn and adapt to new patterns without catastrophic forgetting, making them ideal for real-time systems requiring continuous learning.

Over the years, dozens of ART variations have been published (Silva et al., 2019a), extending the applicability of ART to nearly all learning regimes, including reinforcement learning (Tan, 2004; Tan et al., 2008), hierarchical clustering (Bartfai, 1994), topological clustering (Tscherepanow, 2010), and biclustering (Xu et al., 2012; Xu & II, 2011). These numerous models provide an ART-based solution for most machine learning use cases. However, the rapid pace of bespoke model development, coupled with the challenges students face in learning ART's foundational principles, has contributed to a scarcity of open-source, user-friendly implementations for most ART variants.

The ability of ART to preserve previously learned patterns while learning new data in real-time has made it a powerful tool in domains such as robotics, medical diagnosis, and adaptive control systems. **artlib** aims to extend the application of these models in modern machine learning pipelines, offering a unique and approachable toolkit for leveraging ART's strengths.

Elementary Models

An elementary ART model is a model capable of unsupervised learning and which relies on no subordinate ART model for its behavior. The following elementary ART models are currently implemented:

1. ART1 (Gail A. Carpenter & Grossberg, 1987a): ART1 was the first ART clustering algorithm to be developed. It clusters binary vectors using a similarity metric based on the Hamming distance.

2. ART2 (Gail A. Carpenter et al., 1991a; Gail A. Carpenter & Grossberg, 1987b): ART2 was the first attempt to extend ART1 to the domain of continuous-valued data. ART2-A was developed shortly after and improved the algorithmic complexity while retaining the properties of the original ART2 implementation. ART2 more closely resembles a multi-layer perceptron network as it uses an adaptive weight vector in the activation layer and a Heaviside function for the resonance layer. ART2 is widely considered to not work and is not recommended for use. It is included here for historical purposes.

3. Fuzzy ART (Gail A. Carpenter et al., 1991b): is arguably the most widely used ART variant at this time. Fuzzy ART is a hyper-box based clustering method, capable of clustering continuous-valued data. Data is pre-processed into zero-volume hyper-boxes through the process of complement coding before being used to initialize or expand the volume of a cluster hyper-box. In the fast-learning regime, Fuzzy ART suffers no catastrophic forgetting. It is exceptionally fast and explainable.

4. Hyperpshere ART (Anagnostopoulos & Georgiopulos, 2000): Hypersphere ART was designed to succeed Fuzzy ART with a more efficient internal knowledge representation. Categories are hyper-pspheres and require less internal memory. However, computational complexity is increased relative to Fuzzy ART.

5. Ellipsoid ART(Anagnostopoulos & Georgiopoulos, 2001a, 2001b): Ellipsoid ART is a generalization of Hyperpshere ART which finds ellipsoidal clusters with arbitrary rotation. Ellipsoid ART is highly order-dependent.

6. Guassian ART (Williamson, 1996): Gaussian ART clusters data in Gaussian distributions (Hyper-ellipsoids) and is similar to Bayesian ART but differs in that the hyper-ellipsoid always have their principal axes square to the coordinate frame. It is also faster than Bayesian ART.

7. Bayesian ART (Vigdor & Lerner, 2007): Bayesian ART clusters data in Bayesian Distributions (Hyper-ellipsoids) and is similar to Gaussian ART but differs in that it allows arbitrary rotation of the hyper-ellipsoid.

8. Quadratic Neuron ART (M.-C. Su & Liu, 2001; M. Su & Liu, 2005): QN-ART uses a neural activation and resonance method to create clusters in a hyper-ellipsoid structure. It is superficially similar to ART2-A in that it uses a weight vector but it is more sophisticated in that neurons also learn a bias and quadratic activation term.

Compound Models

A compound ART model is one which extends the functionality of one or more underlying ART models. Functional extension may include topology learning, supervised learning, reinforcement learning, and more. The following compound ART models are currently implemented:

1. ARTMAP (G. A. Carpenter et al., 1991): ARTMAP uses two ART modules to separately cluster two parallel data streams (A and B). An inter-ART module regulates the clustering such that clusters in the `module_A` maintain a many-to-one mapping with the clusters in the `module_B` by using a match-tracking function. When the data stream are independent and dependent variables for the A and B side respectively, ARTMAP learns a functional mapping that describes their relationship. ARTMAP can therefore be used for both classification and regression tasks.

2. Simple ARTMAP (Gotarredona et al., 1998): Simple ARTMAP (or Simplified ARTMAP) was developed to streamline the ARTMAP algorithm for classification tasks. As most classification problems provide discrete labels, it is possible to replace the B-side of

DRAFT

the ARTMAP algorithm with the class labels directly. Simple ARTMAP does this and creates a mapping from B-side class labels to A-side cluster labels. The many-to-one mapping property is preserved, but the learning process is significantly faster and less computationally intensive. Simple ARTMAP is the recommended model for classification tasks.

3. Fusion ART (Tan et al., 2007): Fusion ART allows for the fusion of multi-channel data by leveraging a discrete elementary ART module for each channel. Activation for each category is calculated as a weighted sum of all channel activations and resonance only occurs if all channels are simultaneously resonant. This allows Fusion ART to learn mappings across all channels. Fusion ART works exceptionally well for regression problems and is the recommended model for this task.

4. FALCON (Tan, 2004): The Reactive Fusion Architecture for Learning, Cognition, and Navigation (FALCON) model is a special case of Fusion ART designed for reinforcement learning. A Fusion ART model is used for learning the mapping between state, action, and reward channels. Special functions are implemented for selecting the optimal action and for predicting reward values.

5. TD-FALCON (Tan et al., 2008): Temporal Difference (TD) FALCON is an extension of Reactive FALCON which utilizes the SARSA (State-Action-Reward-State-Action) method for temporal difference reinforcement learning. TD-FALCON is the recommended model for reinforcement learning tasks.

6. Dual Vigilance ART (Silva et al., 2019b): Dual Vigilance ART extends an elementary ART module with a second, less restrictive vigilance parameter. Further, Dual Vigilance ART distinguishes between clusters and category lables by mapping clusters to category labels in a many-to-one fashion. Clusters are formed using the typical process for the underlying art module unless no resonant cluster is found. When this occurs, the less-restrictive vigilance parameter is used to determine if a near-resonant cluster can be found. If one can be found, a new cluster is formed, and the near-resonant cluster's category label is mapped to the new cluster. If neither a resonant nor near-resonant category can be found, a new cluster and new category label are both created and mapped to eachother. In this way, Dual Vigilance ART is capable of finding arbitrarily shaped structures as topological composites of the underlying ART geometry (i.e Hyper-ellipsoids or Hyper-boxes).

7. SMART (Bartfai, 1994): Self-consistent Modular ART (SMART) is a special case of Deep ARTMAP and an extension of ARTMAP. SMART permits n-many modules (in contrast to ARTMAP's 2-modules) which have vigilance parameters monotonically increasing with depth. By passing the same sample vector to all modules, SMART creates a self-consistent hierarchy of clusters through a divisive clustering approach. The number of modules and granularity at each module are both parameterizable.

8. Deep ARTMAP: Deep ARTMAP is a novel contribution of this library. It generalizes SMART by permitting each module `module_i` to accept some function $f^i(x)$. This generalization allows the user to find hierarchical relationships between an abritrary number of functional transformations of some input data. When only two modules are used and $f^1(x) = target(x)$ and $f^2(x) = x$ Deep ARTMAP reduces to standard ARTMAP. Similarly, when the functional transformation at each layer is the identity function, Deep ARTMAP reduces to SMART.

9. Topo ART (Tscherepanow, 2010): Topo ART is a topological clustering approach which uses an elementary ART module to learn a distributed clustering graph where samples can belong to multiple distinct clusters. The co-resonant clusters are tracked using an adjacency matrix which describes the cluster relationships of the entire model.

10. CVI ART (Silva et al., 2022): CVI ART maps the clusters of an elementary ART module to category label identified by the optimal Cluster Validity Index (CV). This mapping

DRAFT

occurs similarly to simplified ARTMAP. An iterative implementation (iCVI ART) is also provided, however it is currently only compatible with Fuzzy ART.

11. BARTMAP (Xu et al., 2012; Xu & II, 2011): BARTMAP is a Biclustering algorithm based loosely on ARTMAP. The algorithm accepts two instantiated elementary ART modules `module_A` and `module_B` which cluster the rows (samples) and columns (features) respectively. The features are clustered independently, but the samples are clustered by considering samples already within a row cluster as well as the candidate sample and enforcing a minimum Pearson correlation within the subset of features belonging to at least one of the feature clusters.

General Usage

To install AdaptiveResonanceLib, simply use pip:

```
pip install artlib
```

Clustering Data with the Fuzzy ART model

```
from artlib import FuzzyART
import numpy as np

# Your dataset
train_X = np.array([...]) # shape (n_samples, n_features)
test_X = np.array([...])

# Initialize the Fuzzy ART model
model = FuzzyART(rho=0.7, alpha = 0.0, beta=1.0)

# Prepare Data
train_X_prep = model.prepare_data(train_X)
test_X_prep = model.prepare_data(test_X)

# Fit the model
model.fit(train_X_prep)

# Predict data labels
predictions = model.predict(test_X_prep)
```

Fitting a Classification Model with SimpleARTMAP

```
from artlib import GaussianART, SimpleARTMAP
import numpy as np

# Your dataset
train_X = np.array([...]) # shape (n_samples, n_features)
train_y = np.array([...]) # shape (n_samples, ), must be integers
test_X = np.array([...])

# Initialize the Gaussian ART model
sigma_init = np.array([0.5]*train_X.shape[1]) # variance estimate for each feature
module_a = GaussianART(rho=0.0, sigma_init=sigma_init)

# Initialize the SimpleARTMAP model
model = SimpleARTMAP(module_a=module_a)

# Prepare Data
```

DRAFT

```
train_X_prep = model.prepare_data(train_X)
test_X_prep = model.prepare_data(test_X)

# Fit the model
model.fit(train_X_prep, train_y)

# Predict data labels
predictions = model.predict(test_X_prep)
```

Fitting a Regression Model with FusionART

```
from artlib import FuzzyART, HypersphereART, FusionART
import numpy as np

# Your dataset
train_X = np.array([...]) # shape (n_samples, n_features_X)
train_y = np.array([...]) # shape (n_samples, n_features_y)
test_X = np.array([...])

# Initialize the Fuzzy ART model
module_x = FuzzyART(rho=0.0, alpha = 0.0, beta=1.0)

# Initialize the Hypersphere ART model
r_hat = 0.5*np.sqrt(train_X.shape[1]) # no restriction on hyperpshere size
module_y = HypersphereART(rho=0.0, alpha = 0.0, beta=1.0, r_hat=r_hat)

# Initialize the FusionARTMAP model
gamma_values = [0.5, 0.5] # eqaul weight to both channels
channel_dims = [
  2*train_X.shape[1], # fuzzy ART complement codes data so channel dim is 2*n_features
  train_y.shape[1]
]
model = FusionART(
  modules=[module_x, module_y],
  gamma_values=gamma_values,
  channel_dims=channel_dims
)

# Prepare Data
train_Xy = model.join_channel_data(channel_data=[train_X, train_y])
train_Xy_prep = model.prepare_data(train_Xy)
test_Xy = model.join_channel_data(channel_data=[train_X], skip_channels=[1])
test_Xy_prep = model.prepare_data(test_Xy)

# Fit the model
model.fit(train_X_prep, train_y)

# Predict y-channel values
pred_y = model.predict_regression(test_Xy_prep, target_channels=[1])
```

Statement of Need

The Adaptive Resonance Library (**artlib**) is essential for researchers, developers, and educators interested in adaptive neural networks, specifically ART algorithms. While deep learning

DRAFT

dominates machine learning, ART models offer unique advantages in incremental and real-time learning environments due to their ability to learn new data without forgetting previously learned information.

Currently, no comprehensive Python library implements a variety of ART models in an open-source, modular, and extensible manner. **artlib** fills this gap by offering a range of ART implementations that integrate seamlessly with machine learning workflows, including scikit-learn's `Pipeline` and `GridSearchCV` (Pedregosa et al., 2011). The library is designed for ease of use and high performance, leveraging Python's scientific stack (NumPy (C. R. Harris et al., 2020), SciPy (Virtanen et al., 2020), and scikit-learn (Pedregosa et al., 2011)) for fast numerical computation.

The modular design of **artlib** enables users to create novel compound ART models, such as Dual Vigilance Fusion ART (Silva et al., 2019b; Tan et al., 2007) or Quadratic Neuron SMART (Bartfai, 1994; M.-C. Su & Liu, 2001; M. Su & Liu, 2005). This flexibility offers powerful experimental and time-saving benefits, allowing researchers and practitioners to evaluate models on diverse datasets efficiently.

Additionally, the library serves as a valuable educational tool, providing well-documented code and familiar APIs to support hands-on experimentation with ART models. It is ideal for academic courses or personal projects in artificial intelligence and machine learning, making **artlib** a versatile resource.

artlib is actively maintained and designed for future extension, allowing users to create new ART models, adjust parameters for specific applications, and explore ART's potential for novel research problems. Its integration with popular Python libraries ensures its adaptability to current machine learning challenges.

Comparison to Existing Implementations

While there are several open-source repositories that provide python implementations of specific ART models (AI OpenLab, 2023; Birkjohann, 2023; Dilekman, 2022; Dixit, 2020; Inyoot, 2021; Ray11641, 2023; Stepanov, 2022; Valixandra, 2021; Wan, 2022), they lack modularity and are limited in scope, often implementing just one or two models. For instance, MATLAB-based ART toolboxes (Boitet, 2022; M. Harris, 2000; Schwenker, 2004b, 2004a) provide implementations of Fuzzy ART, TopoART, ART1, and ARTMAP models, but they lack the flexibility and modularity required for broader experimentation. The most significant existing ART implementation exists in julia and provides just five models (Petrenko & Wunsch, 2022) but, like the previously listed MATLAB-based toolboxes, it is not easily accessible to Python-based work flows and lacks a modular design.

These existing implementations of ART models may provide standalone versions of individual models, but they are often not designed to integrate seamlessly with modern Python libraries such as scikit-learn, NumPy, and SciPy. As a result, researchers and developers working in Python-based environments face challenges when trying to incorporate ART models into their machine learning pipelines.

In contrast, **artlib** offers a comprehensive and modular collection of ART models, including both elementary and compound ART architectures. It is designed for interoperability with popular Python tools, enabling users to easily integrate ART models into machine learning workflows, optimize models using scikit-learn's `GridSearchCV`, and preprocess data using standard libraries. Further, **artlib** provides users the flexibility to construct their own compound ART modules (those art modules deriving properties from other, elementary modules) which may or may not exist in published literature. **artlib** also provides a template in the source code to encourage users to develop and experiment with their own custom ART algorithms. This flexibility and integration make **artlib** a powerful resource for both research and practical applications.

Acknowledgements

This research was supported by the National Science Foundation (NSF) under Award Number 2420248. The project titled EAGER: Solving Representation Learning and Catastrophic Forgetting with Adaptive Resonance Theory provided essential funding for the completion of this work.

We would also like to thank Gail Carpenter and Stephen Grossberg for their feedback regarding this project and their immense contribution to machine learning by pioneering Adaptive Resonance Theory.

References

AI OpenLab. (2023). *ART: Adaptive resonance theory implementation* (Version v1.0.0). https://github.com/AIOpenLab/art. https://github.com/AIOpenLab/art

Anagnostopoulos, G. C., & Georgiopoulos, M. (2001a). Ellipsoid ART and ARTMAP for incremental clustering and classification. *IJCNN'01. International Joint Conference on Neural Networks. Proceedings (Cat. No. 01CH37222)*, *2*, 1221–1226.

Anagnostopoulos, G. C., & Georgiopoulos, M. (2001b). Ellipsoid ART and ARTMAP for incremental unsupervised and supervised learning. *Applications and Science of Computational Intelligence IV*, *4390*, 293–304.

Anagnostopoulos, G. C., & Georgiopulos, M. (2000). Hypersphere ART and ARTMAP for unsupervised and supervised, incremental learning. *Proceedings of the IEEE-INNS-ENNS International Joint Conference on Neural Networks. IJCNN 2000. Neural Computing: New Challenges and Perspectives for the New Millennium*, *6*, 59–64.

Bartfai, G. (1994). Hierarchical clustering with ART neural networks. *Proceedings of 1994 IEEE International Conference on Neural Networks (ICNN'94)*, *2*, 940–944.

Bezdek, J. C., & Hathaway, R. J. (2002). VAT: A tool for visual assessment of (cluster) tendency. *Proceedings of the 2002 International Joint Conference on Neural Networks. IJCNN'02 (Cat. No. 02CH37290)*, *3*, 2225–2230.

Birkjohann, C. (2023). *Art-python: Adaptive resonance theory in python* (Version v0.1.0). https://github.com/cbirkj/art-python. https://github.com/cbirkj/art-python

Boitet, M. (2022). *TopoART neural networks*. https://www.mathworks.com/matlabcentral/fileexchange/118455-topoart-neural-networks. https://www.mathworks.com/matlabcentral/fileexchange/118455-topoart-neural-networks

Carpenter, Gail A., & Grossberg, S. (1987a). A massively parallel architecture for a self-organizing neural pattern recognition machine. *Computer Vision, Graphics, and Image Processing*, *37*(1), 54–115.

Carpenter, Gail A., & Grossberg, S. (1987b). ART 2: Self-organization of stable category recognition codes for analog input patterns. *Applied Optics*, *26*(23), 4919–4930.

Carpenter, G. A., Grossberg, S., & Reynolds, J. H. (1991). ARTMAP: Supervised real-time learning and classification of nonstationary data by a self-organizing neural network. *[1991 Proceedings] IEEE Conference on Neural Networks for Ocean Engineering*, 341–342. https://doi.org/10.1109/ICNN.1991.163370

Carpenter, Gail A., Grossberg, S., & Rosen, D. B. (1991a). ART 2-a: An adaptive resonance algorithm for rapid category learning and recognition. *Neural Networks*, *4*(4), 493–504.

Carpenter, Gail A., Grossberg, S., & Rosen, D. B. (1991b). Fuzzy ART: Fast stable learning and categorization of analog patterns by an adaptive resonance system. *Neural Networks*,

DRAFT

4(6), 759–771.

Dilekman, K. (2022). *Artificial neural network - adaptive resonance theory* (Version v0.1.0). https://github.com/KeremDlkmn/artificial-neural-network-adaptive-resonance-theory. https://github.com/KeremDlkmn/artificial-neural-network-adaptive-resonance-theory

Dixit, M. (2020). *Adaptive resonance theory* (Version v1.0.0). https://github.com/MeetiDixit/AdaptiveResonanceTheory. https://github.com/MeetiDixit/AdaptiveResonanceTheory

Gotarredona, T. S., Barranco, B. L., & Andreou, A. (1998). *Adaptive resonance theory microchips circuit design techniques*. Norwell, MA: Kluwer.

Grossberg, S. (1976). Adaptive pattern classification and universal recoding: I. Parallel development and coding of neural feature detectors. *Biological Cybernetics*, *23*(3), 121–134.

Grossberg, S. (1980). How does a brain build a cognitive code? *Psychological Review*, *87 1*, 1–51.

Grossberg, S. (2013). Adaptive resonance theory: How a brain learns to consciously attend, learn, and recognize a changing world. *Neural Networks*, *37*, 1–47.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Harris, M. (2000). *ART1S: Adaptive resonance theory 1 (ART1)*. https://www.mathworks.com/matlabcentral/fileexchange/93-art1s-zip. https://www.mathworks.com/matlabcentral/fileexchange/93-art1s-zip

Inyoot. (2021). *ART: Adaptive resonance theory implementation in python* (Version v0.1.0). https://github.com/inyoot/art. https://github.com/inyoot/art

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Petrenko, S., & Wunsch, D., II. (2022). AdaptiveResonance.jl: A Julia Implementation of Adaptive Resonance Theory (ART) Algorithms. *Journal of Open Source Software*, *7*. https://doi.org/10.21105/joss.03671

Ray11641. (2023). *Artpy: Adaptive resonance theory in python* (Version v0.1.0). https://github.com/Ray11641/artpy. https://github.com/Ray11641/artpy

Schwenker, F. (2004a). *ART1, fuzzy ART, ARTMAP, fuzzy ARTMAP neural networks*. https://www.mathworks.com/matlabcentral/fileexchange/3070-art1-fuzzyart-artmap-fuzzyartmap. https://www.mathworks.com/matlabcentral/fileexchange/3070-art1-fuzzyart-artmap-fuzzyartmap

Schwenker, F. (2004b). *Fuzzy ART and fuzzy ARTMAP neural networks*. https://www.mathworks.com/matlabcentral/fileexchange/4306-fuzzy-art-and-fuzzy-artmap-neural-networks. https://www.mathworks.com/matlabcentral/fileexchange/4306-fuzzy-art-and-fuzzy-artmap-neural-n

Silva, L. E. B. da, Elnabarawy, I., & Wunsch II, D. C. (2019a). A survey of adaptive resonance theory neural network models for engineering applications. *Neural Networks*, *120*, 167–203.

Silva, L. E. B. da, Elnabarawy, I., & Wunsch II, D. C. (2019b). Dual vigilance fuzzy adaptive resonance theory. *Neural Networks*, *109*, 1–5.

Silva, L. E. B. da, Rayapati, N., & Wunsch, D. C. (2022). iCVI-ARTMAP: Using incremental cluster validity indices and adaptive resonance theory reset mechanism to accelerate validation and achieve multiprototype unsupervised representations. *IEEE Transactions on*

DRAFT

Neural Networks and Learning Systems, *34*(12), 9757–9770.

Stepanov, S. (2022). *ARTpy: A python implementation of adaptive resonance theory* (Version v1.0.0). https://github.com/DeadAt0m/ARTpy. https://github.com/DeadAt0m/ARTpy

Su, M.-C., & Liu, T.-K. (2001). Application of neural networks using quadratic junctions in cluster analysis. *Neurocomputing*, *37*(1-4), 165–175.

Su, M., & Liu, Y. (2005). A new approach to clustering data with arbitrary shapes. *Pattern Recognition*, *38*(11), 1887–1901.

Tan, A.-H. (2004). FALCON: A fusion architecture for learning, cognition, and navigation. *2004 IEEE International Joint Conference on Neural Networks (IEEE Cat. No. 04CH37541)*, *4*, 3297–3302.

Tan, A.-H., Carpenter, G. A., & Grossberg, S. (2007). Intelligence through interaction: Towards a unified theory for learning. *International Symposium on Neural Networks*, 1094–1103.

Tan, A.-H., Lu, N., & Xiao, D. (2008). Integrating temporal difference methods and self-organizing neural networks for reinforcement learning with delayed evaluative feedback. *IEEE Transactions on Neural Networks*, *19*(2), 230–244.

Tscherepanow, M. (2010). TopoART: A topology learning hierarchical ART network. *International Conference on Artificial Neural Networks*, 157–167.

Valixandra, F. (2021). *Adaptive resonance theory* (Version v1.0.0). https://github.com/flppvalxndra/Adaptive-Resonance-Theory. https://github.com/flppvalxndra/Adaptive-Resonance-Theory

Vigdor, B., & Lerner, B. (2007). The bayesian artmap. *IEEE Transactions on Neural Networks*, *18*(6), 1628–1644.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Wan, Y. (2022). *ART2: Adaptive resonance theory 2 implementation* (Version v1.0.0). https://github.com/YuChangWan/ART2. https://github.com/YuChangWan/ART2

Williamson, J. R. (1996). Gaussian ARTMAP: A neural network for fast incremental learning of noisy multidimensional maps. *Neural Networks*, *9*(5), 881–897.

Xu, R., & II, D. C. W. (2011). BARTMAP: A viable structure for biclustering. *Neural Networks*, *24*, 709–716. https://doi.org/10.1016/j.neunet.2011.03.020

Xu, R., II, D. C. W., & Kim, S. Y. (2012). *Methods and systems for biclustering algorithm* (Patent No. US9,043,326B2). Filed January 28, 2012, issued May 26, 2015. https://patents.google.com/patent/US9043326B2/en

DRAFT